Sorting II - Heap sort

Shahriar Ivan
Lecturer, Department of CSE, IUT

Heap Sort

Recall that inserting n objects into a min-heap and then taking n objects will result in them coming out in order

Strategy: given an unsorted list with n objects, place them into a heap, and take them out

Run time Analysis of Heap Sort

Taking an object out of a heap with n items requires O(ln(n)) time

Therefore, taking n objects out requires

$$\sum_{k=1}^{n} \ln(k) = \ln\left(\prod_{k=1}^{n} k\right) = \ln(n!)$$

Recall that ln(a) + ln(b) = ln(ab)

Question: What is the asymptotic growth of ln(n!)?

Run time Analysis of Heap Sort

Using Maple:

> asympt(ln(n!), n);

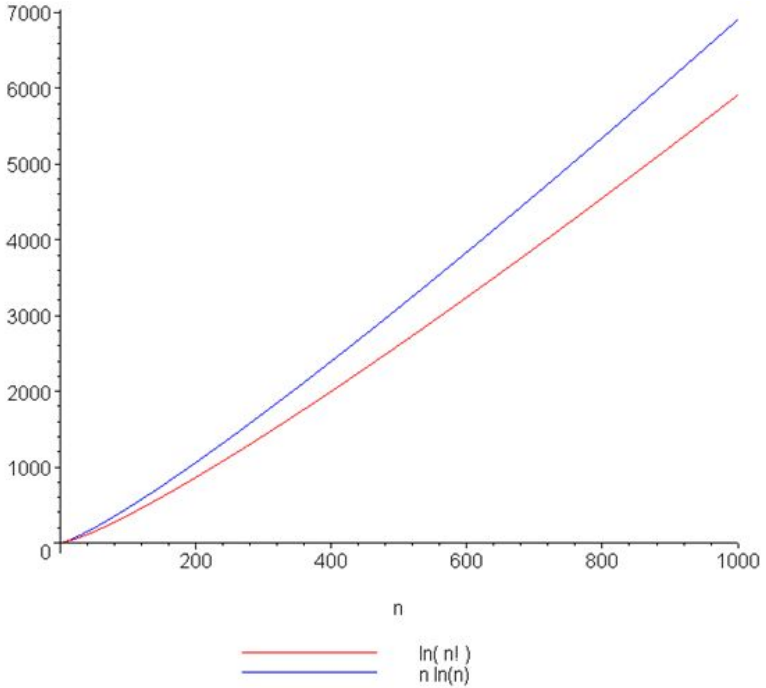
$$(\ln(n) - 1)\, n + \ln(\sqrt{2}\,\sqrt{\pi}) + \frac{1}{2}\ln(n) + \frac{1}{12\,n} - \frac{1}{360\,n^3} + \mathrm{O}\left(\frac{1}{n^5}\right)$$

The leading term is (ln(n) – 1) n

Therefore, the run time is O(n ln(n))

Runtime Analysis of Heap Sort

A plot of ln(n!) and n ln(n) also suggests that they are asymptotically related:

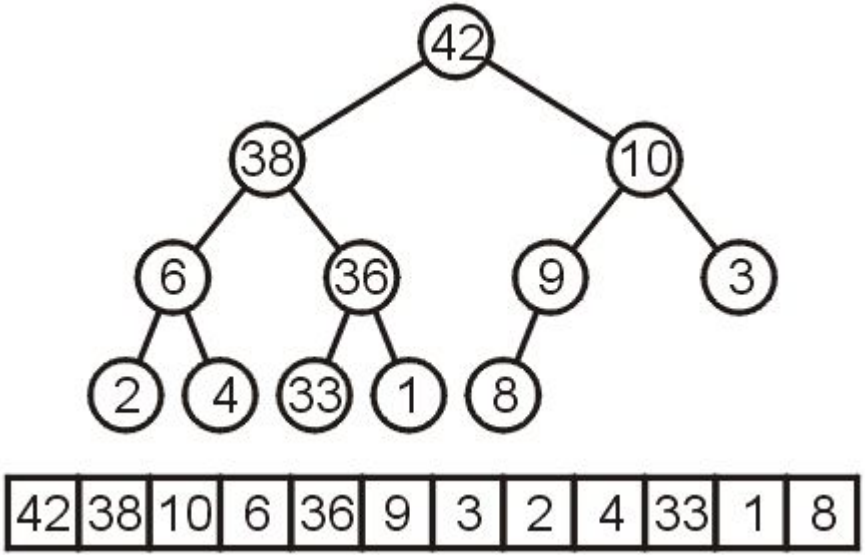


In-place Implementation

- Problem with current method of heap sort
  - This solution requires additional memory, that is, a min-heap of size n
- This requires $\Theta(n)$ memory and is therefore not in place
- Is it possible to perform a heap sort in place, that is, require at most $\Theta(1)$ memory (a few extra variables)?

In-place Implementation

- Instead of implementing a min-heap, consider a max-heap:
- A heap where the maximum element is at the top of the heap and the next to be popped

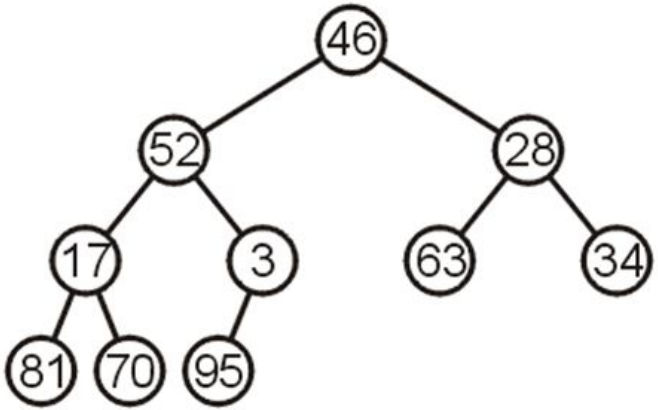


In-place Implementation

Now, consider this unsorted array:

46	52	28	17	3	63	34	81	70	95

This array represents the following complete tree:



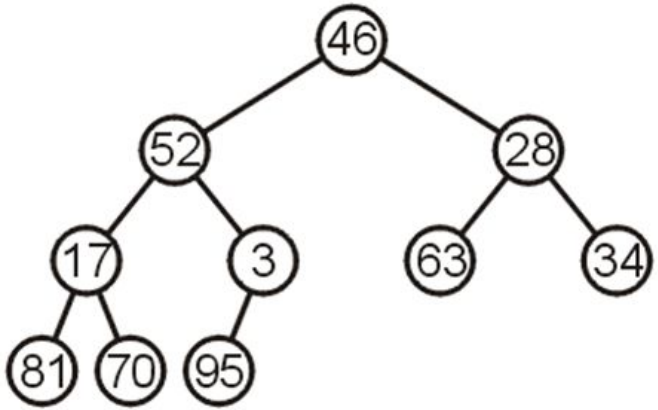
This is neither a min-heap, max-heap, or binary search tree

In-place Implementation

Now, consider this unsorted array:

46	52	28	17	3	63	34	81	70	95

Additionally, because arrays start at 0 (we started at entry 1 for binary heaps) , we need different formulas for the children and parent



The formulas are now:

Children $2*k + 1,\ \ 2*k + 2$

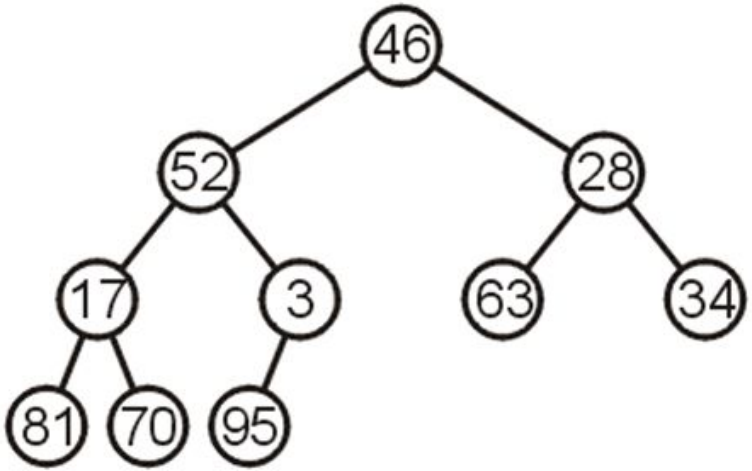
Parent $(k + 1)/2 - 1$

In place Heapification

Can we convert this complete tree into a max heap?

Restriction:

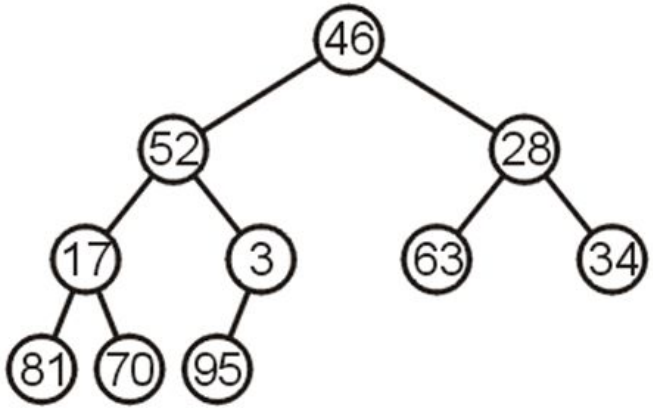
- The operation must be done in-place



In place Heapification

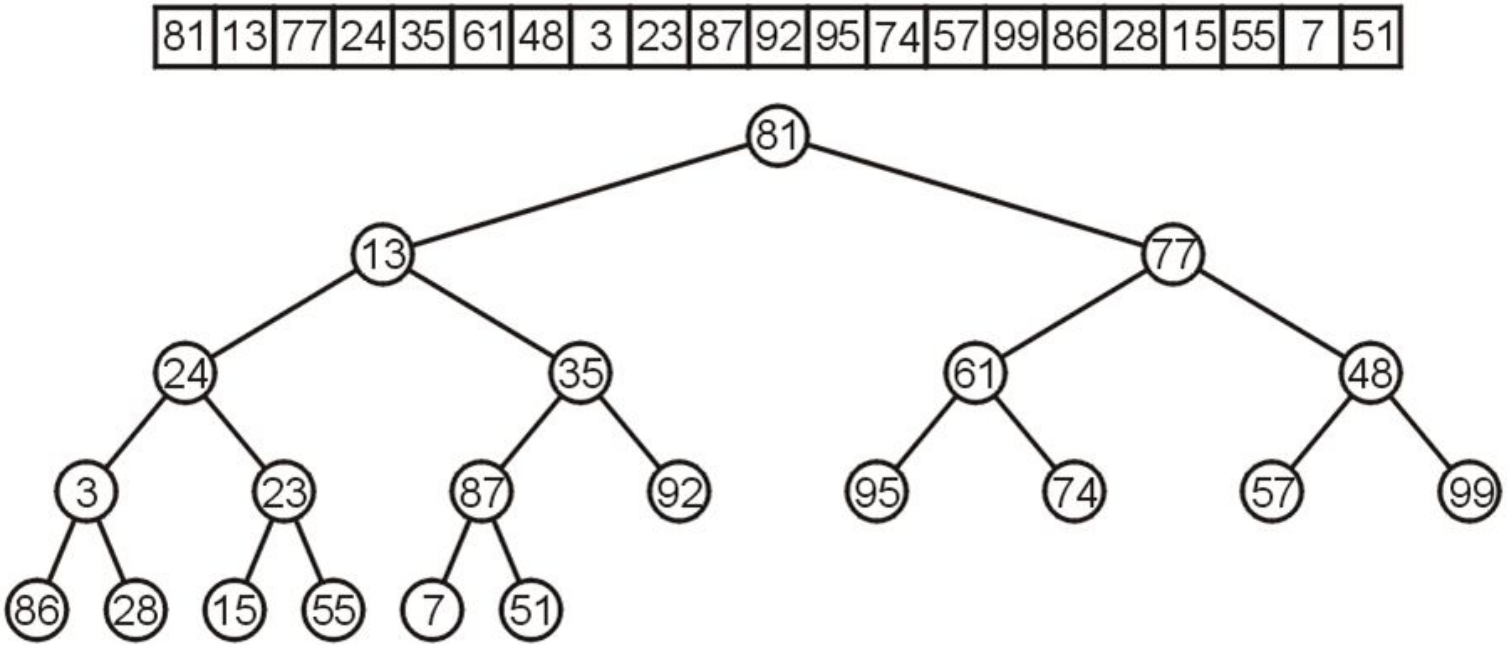
Two strategies:

- Assume 46 is a max-heap and keep inserting the next element into the existing heap (similar to the strategy for insertion sort)
- Start from the back: note that all leaf nodes are already max heaps, and then make corrections so that previous nodes also form max heaps



In place Heapification

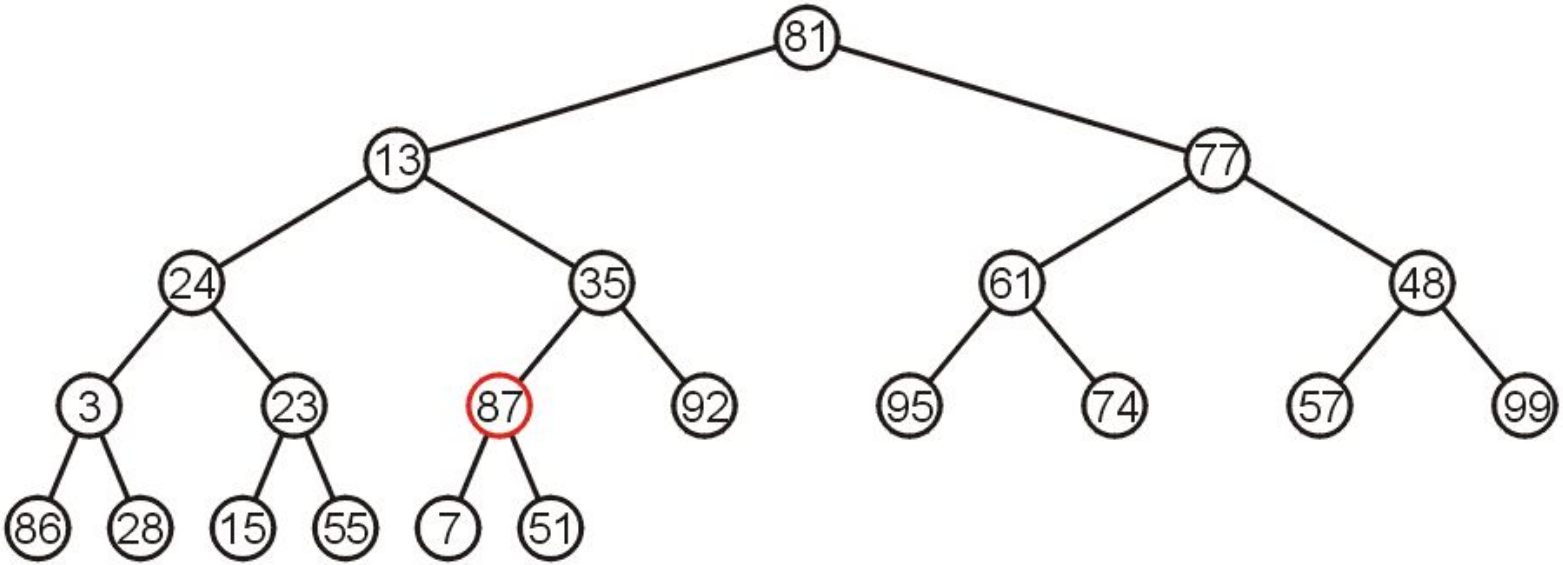
Let’s work bottom-up: each leaf node is a max heap on its own



In place Heapification

Starting at the back, we note that all leaf nodes are trivial heaps

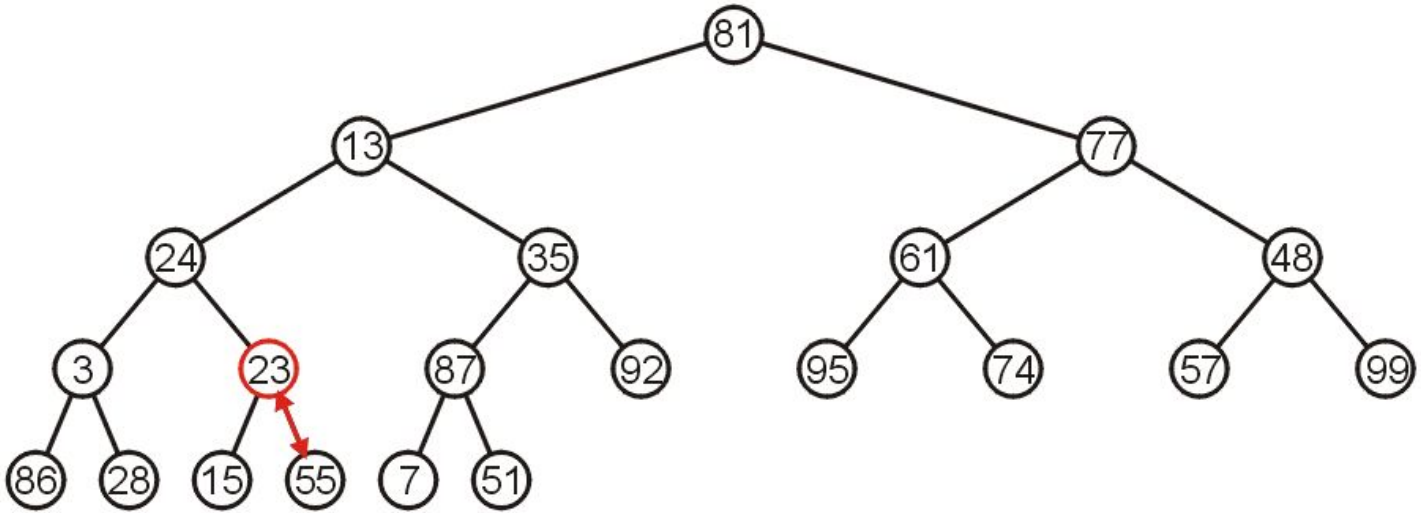
Also, the subtree with 87 as the root is a max-heap



In place Heapification

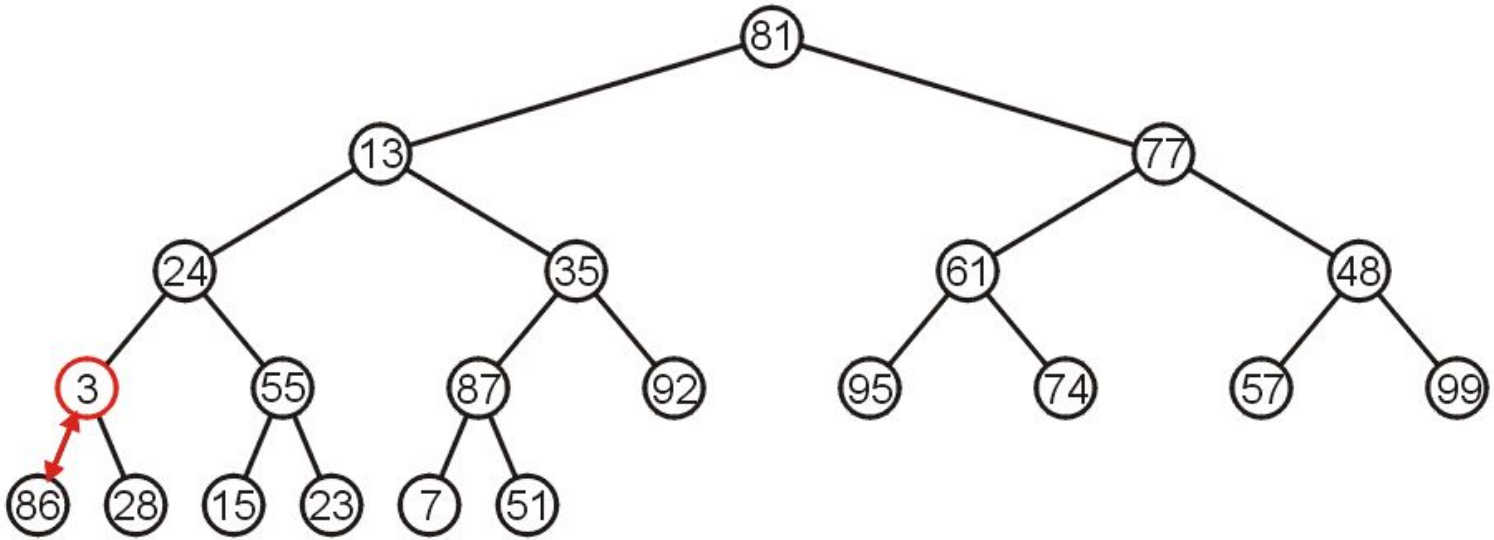
The subtree with 23 is not a max-heap, but swapping it with 55 creates a max-heap

This process is termed percolating down



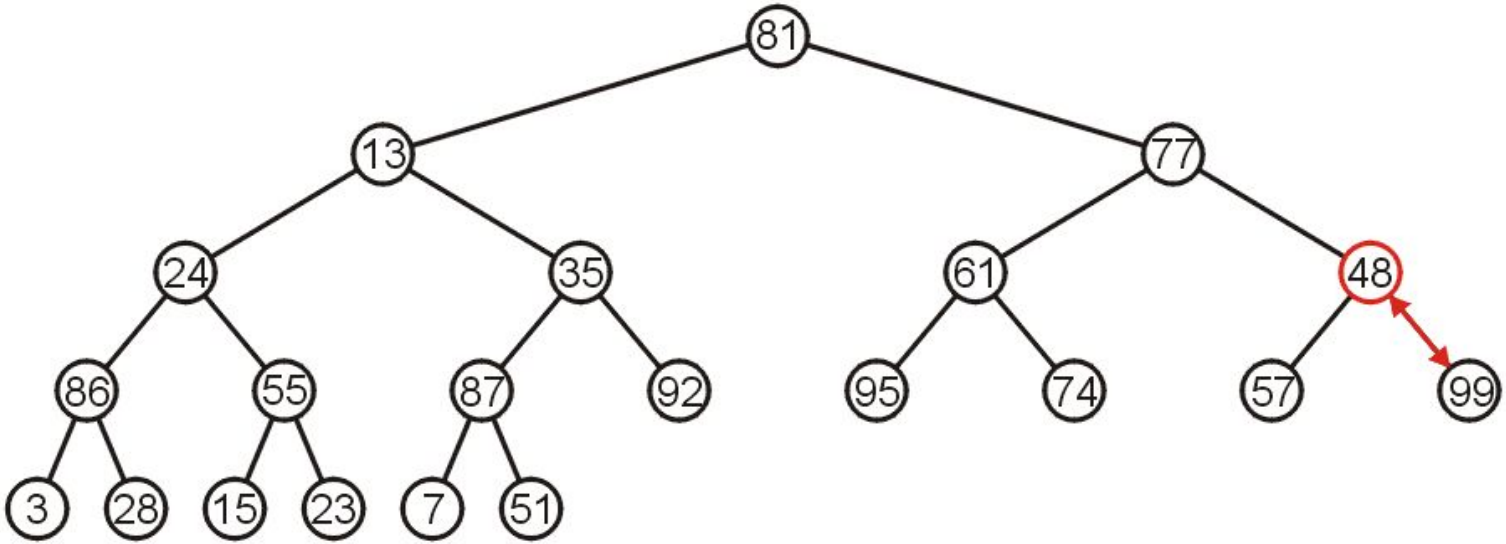
In place Heapification

The subtree with 3 as the root is not max-heap, but we can swap 3 and the maximum of its children: 86



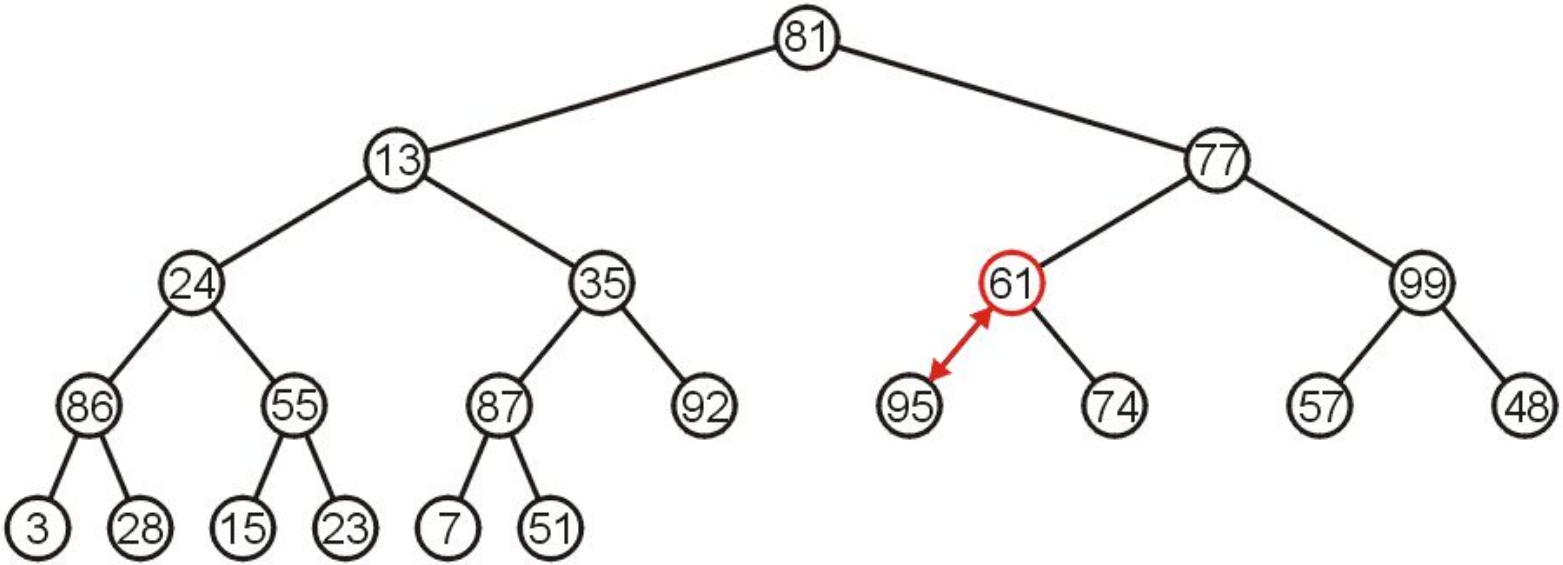
In place Heapification

Starting with the next higher level, the subtree with root 48 can be turned into a max-heap by swapping 48 and 99



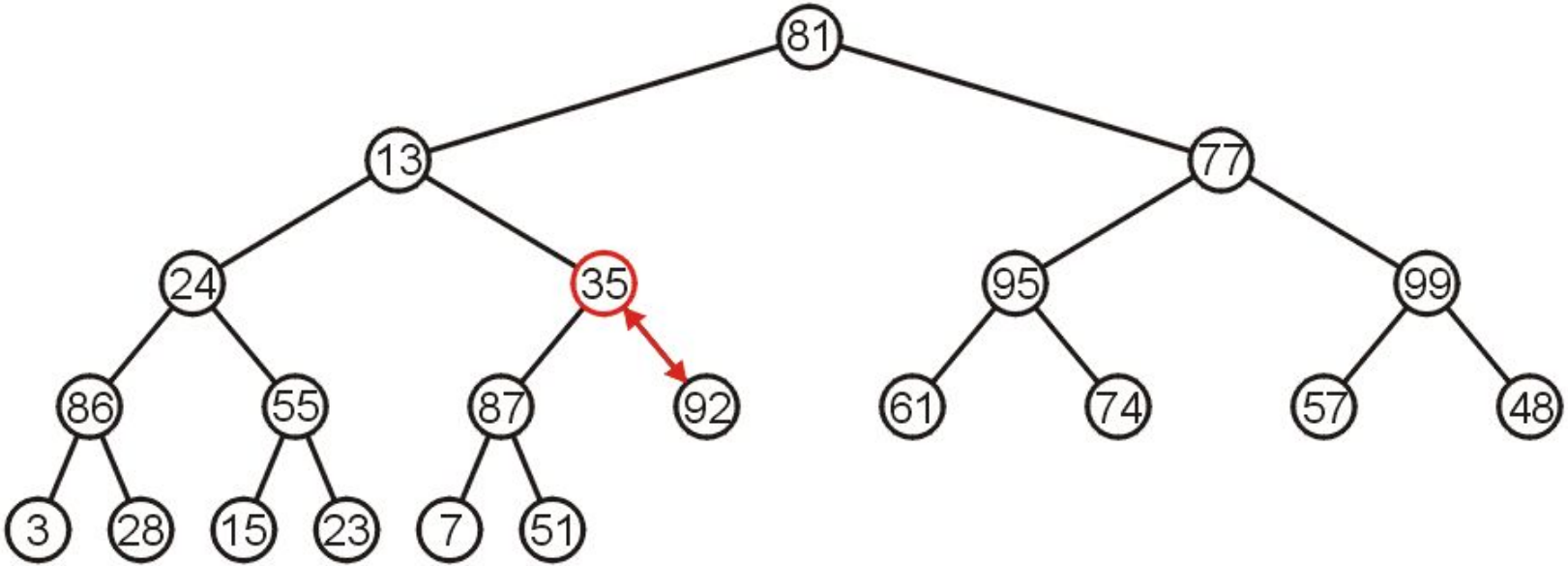
In place Heapification

Similarly, swapping 61 and 95 creates a max-heap of the next subtree



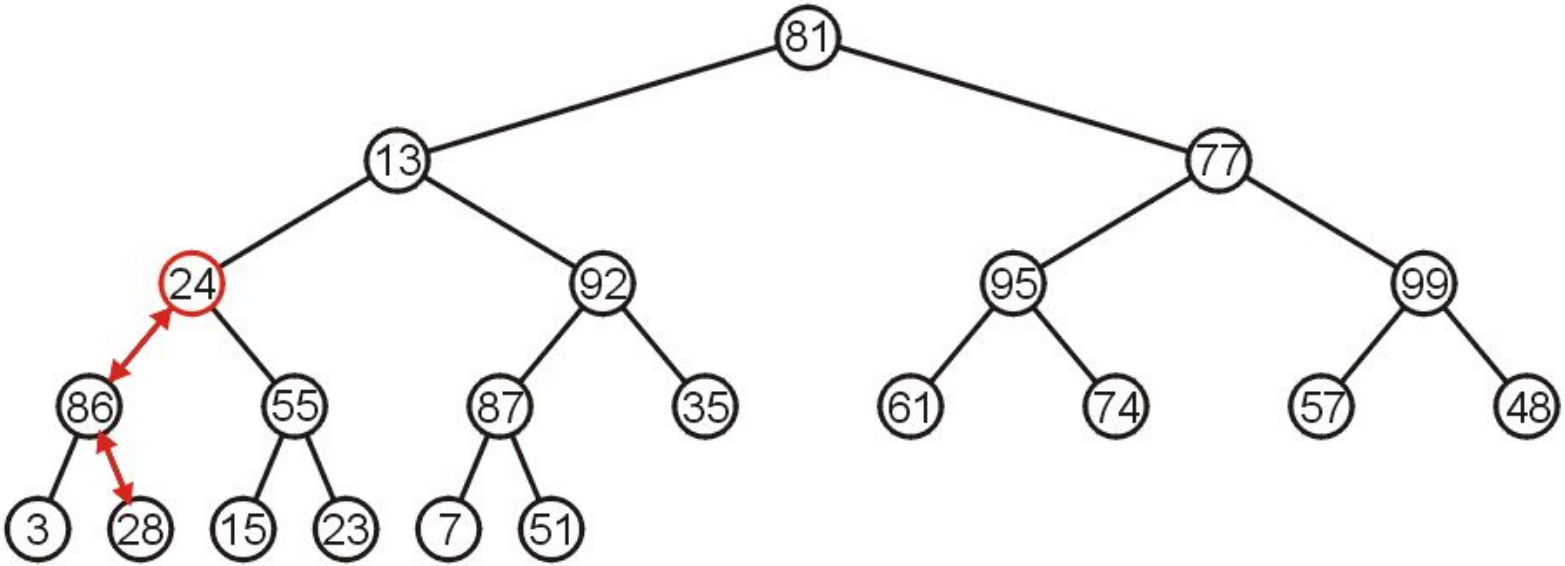
In place Heapification

As does swapping 35 and 92



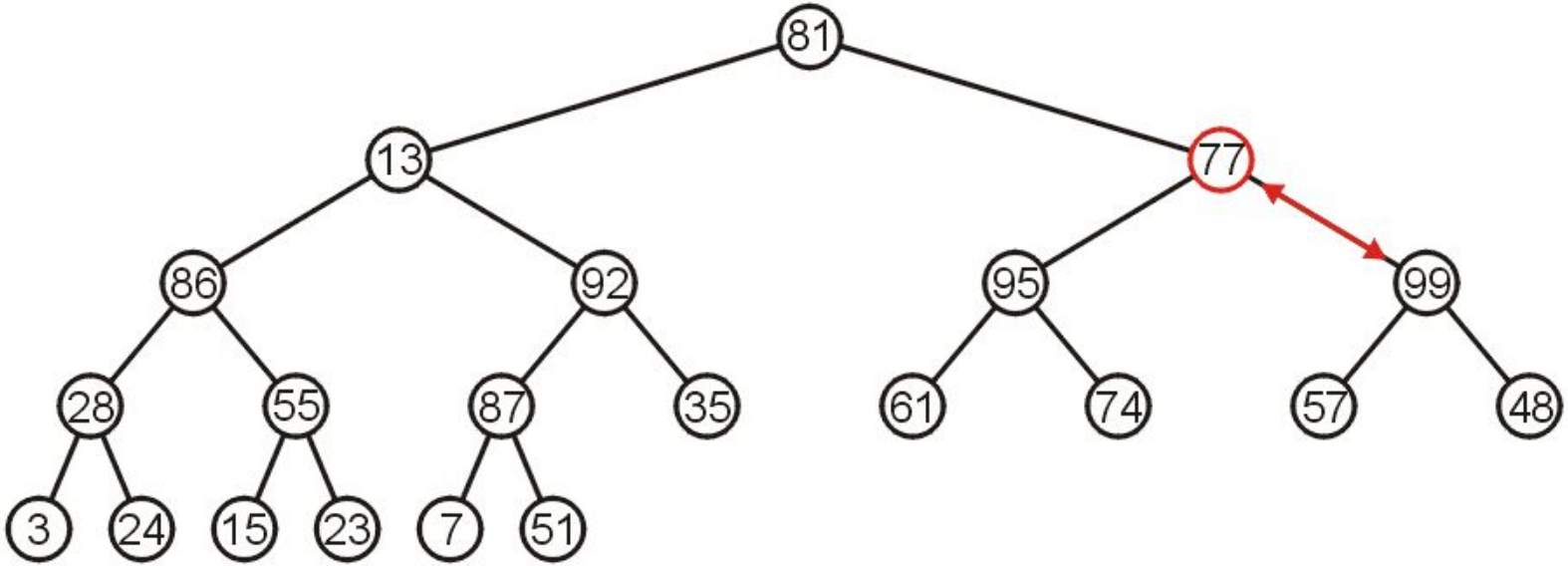
In place Heapification

The subtree with root 24 may be converted into a max-heap by first swapping 24 and 86 and then swapping 24 and 28



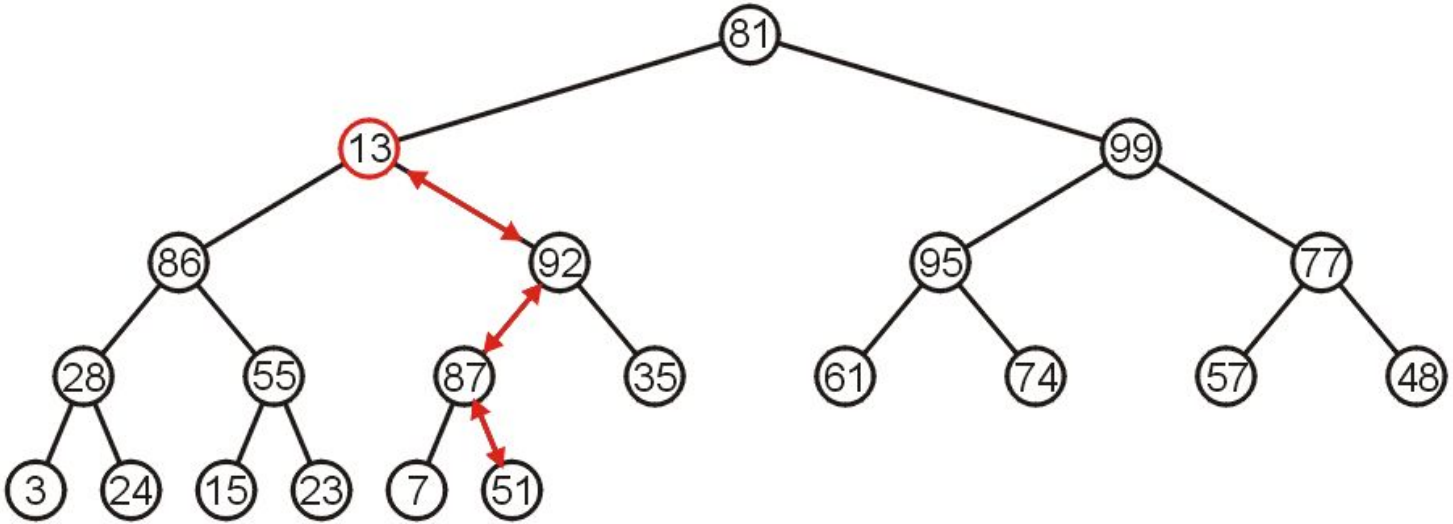
In place Heapification

The right-most subtree of the next higher level may be turned into a max-heap by swapping 77 and 99



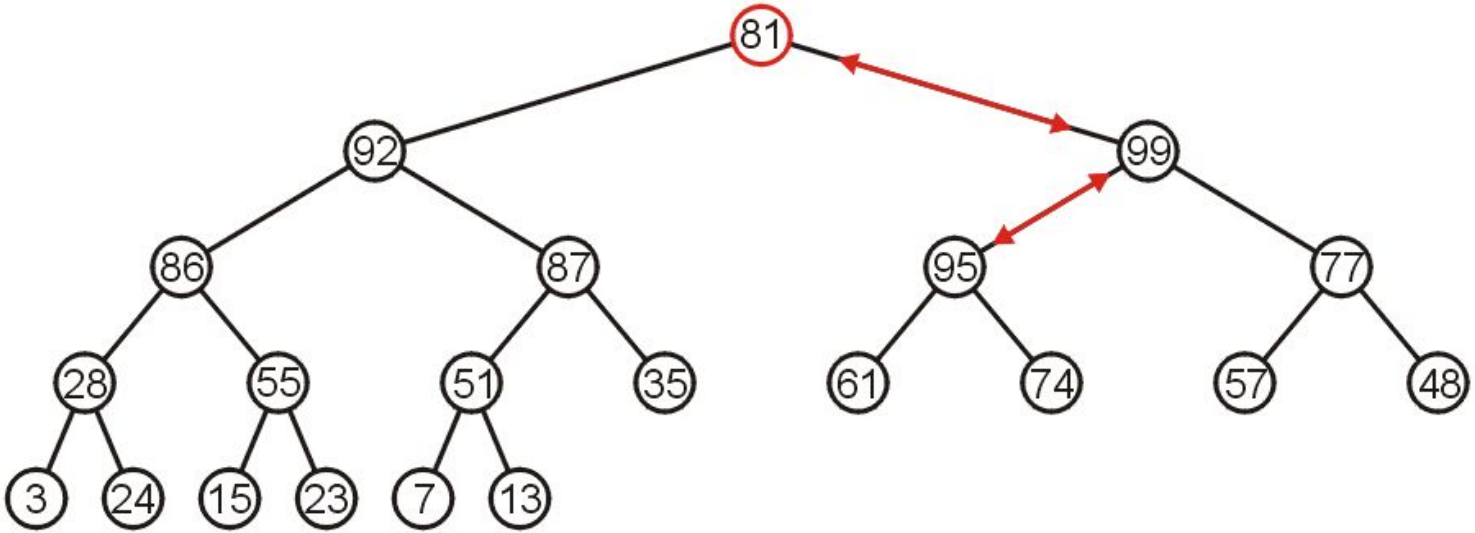
In place Heapification

However, to turn the next subtree into a max-heap requires that 13 be percolated down to a leaf node



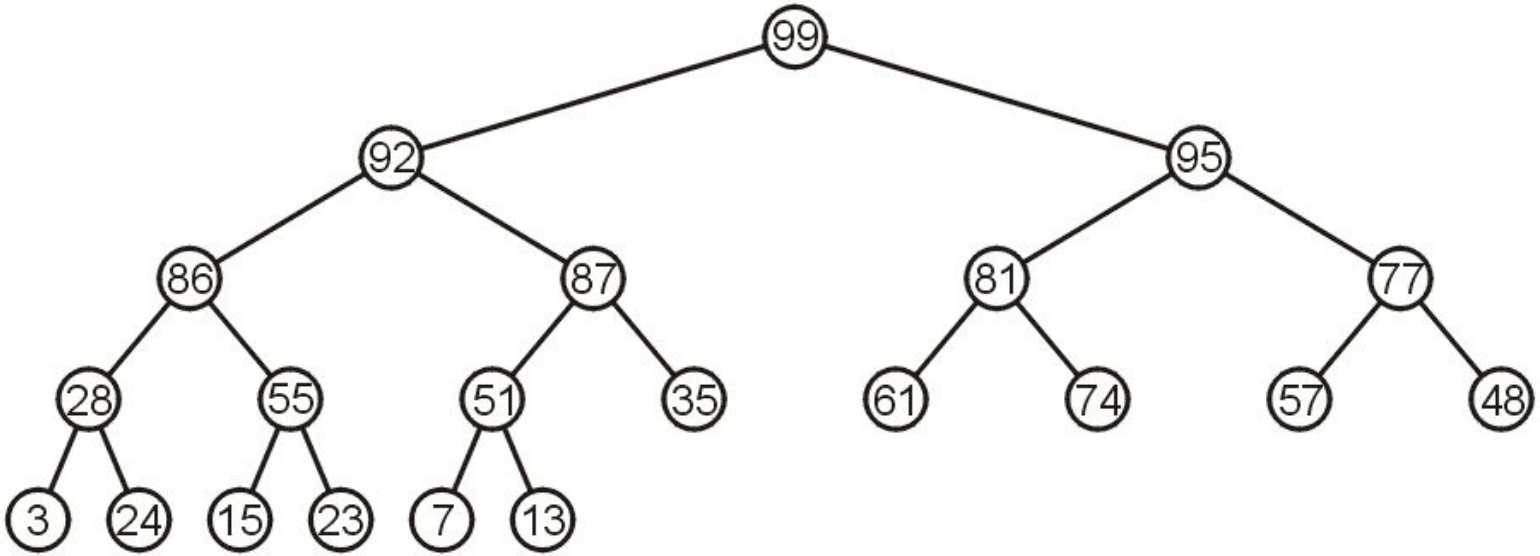
Runtime Analysis of Heapify

The root need only be percolated down by two levels



Runtime Analysis of Heapify

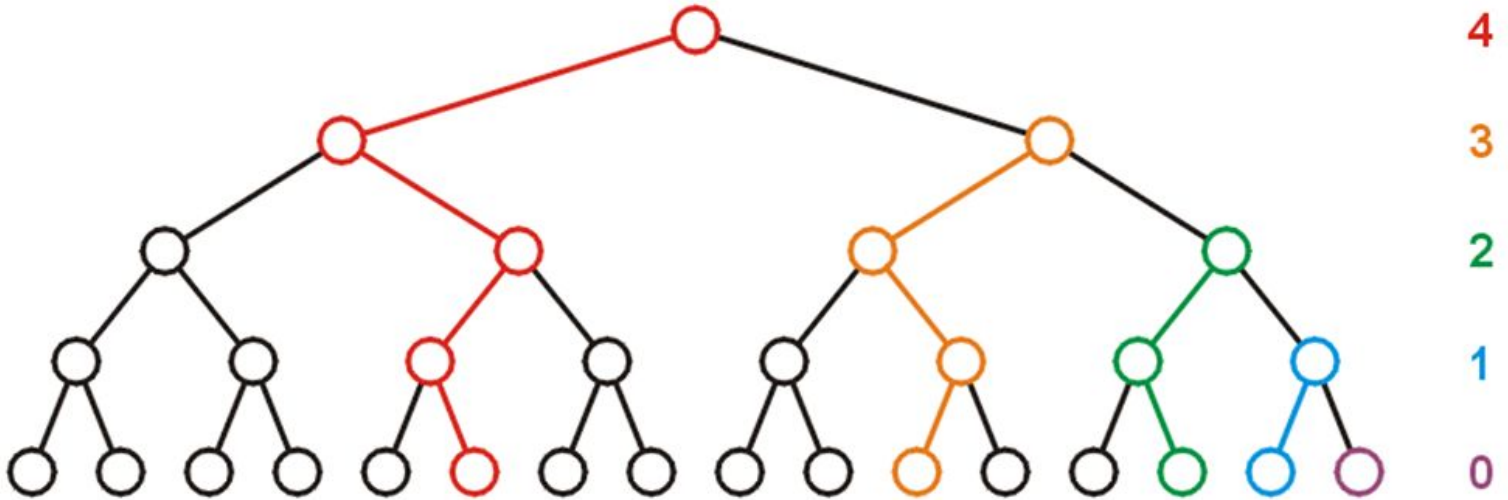
The final product is a max-heap



Runtime Analysis of Heapify

Considering a perfect tree of height h:

The maximum number of swaps which a second-lowest level would experience is 1, the next higher level, 2, and so on



Runtime Analysis of Heapify

At depth k, there are 2^k nodes and in the worst case, all of these nodes would have to percolated down h – k levels

In the worst case, this would requiring a total of $2^k(h - k)$ swaps

Writing this sum mathematically, we get:

$$\sum_{k=0}^{h} 2^k(h-k) = \left(2^{h+1} - 1\right) - (h+1)$$

Runtime Analysis of Heapify

Recall that for a perfect tree, n = 2h + 1 – 1 and h + 1 = lg(n + 1), therefore

$$\sum_{k=0}^{h} 2^k (h-k) = n - \lg(n+1)$$

Each swap requires two comparisons (which child is greatest), so there is a maximum of 2n (or Θ(n)) comparisons

Runtime Analysis of Heapify

Note that if we go the other way (treat the first entry as a max heap and then continually insert new elements into that heap, the run time is at worst

$$\sum_{k=0}^{h} 2^k k = 2^{h+1}(h-1)+2$$

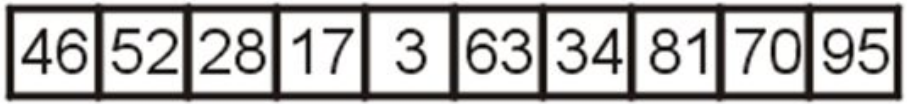
$$= \left(2^{h+1}+1\right)(h-1)-(h-1)+2$$

$$= n\left(\lg(n+1)-2\right)-\lg(n+1)+4 = \Theta\left(n\ln(n)\right)$$

It is significantly better to start at the back

Heap Sort Example

Let us look at this example: we must convert the unordered array with n = 10 elements into a max-heap

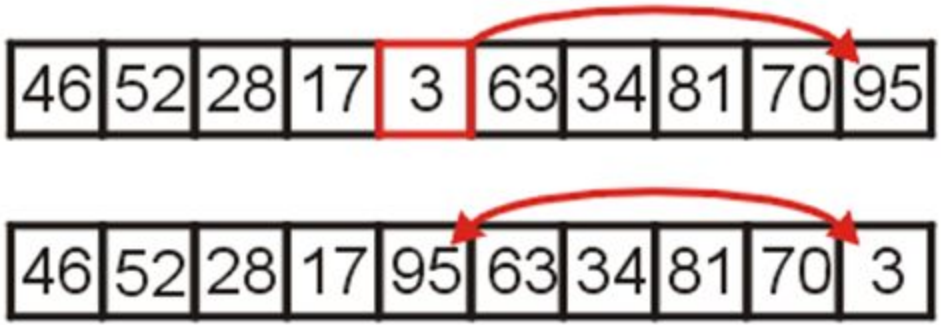


None of the leaf nodes need to be percolated down, and the first non-leaf node is in position n/2

Thus we start with position 10/2 = 5

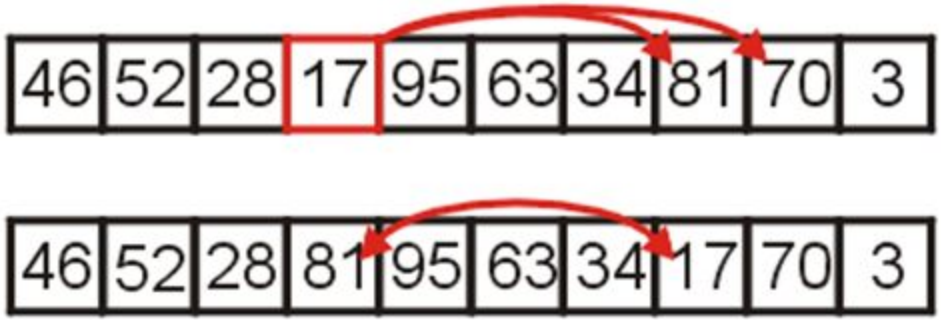
Heap Sort Example

We compare 3 with its child and swap them



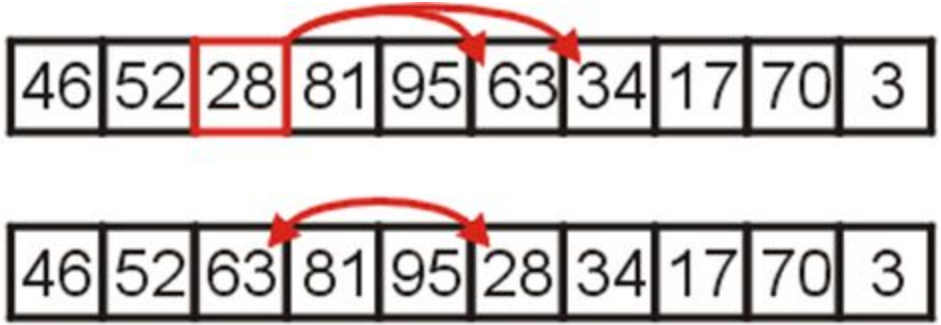
Heap Sort Example

We compare 17 with its two children and swap it with the maximum child (70)



Heap Sort Example

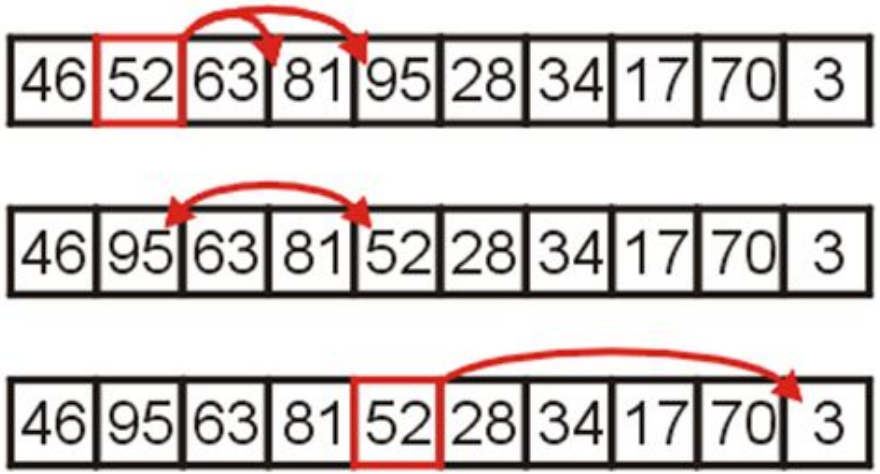
We compare 28 with its two children, 63 and 34, and swap it with the largest child



Heap Sort Example

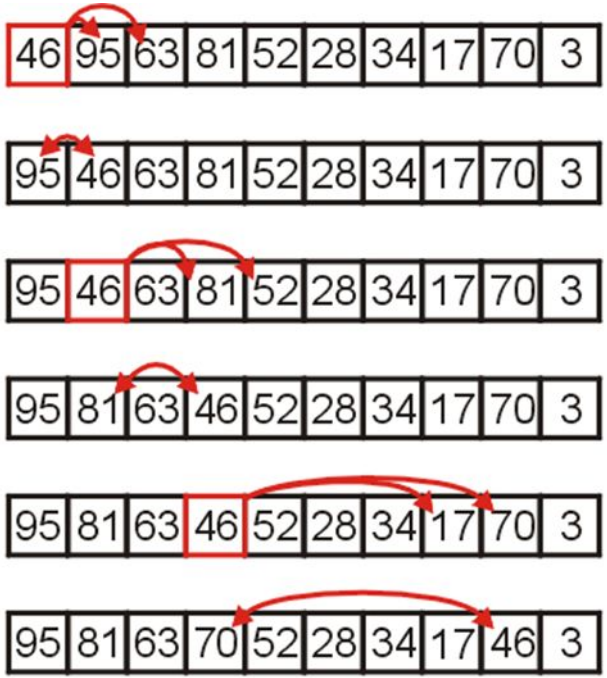
We compare 52 with its children, swap it with the largest

Recursing, no further swaps are needed



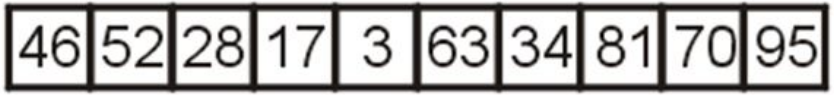
Heap Sort Example

Finally, we swap the root with its largest child, and recurse, swapping 46 again with 81, and then again with 70

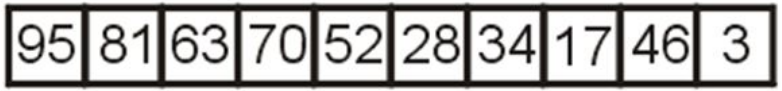


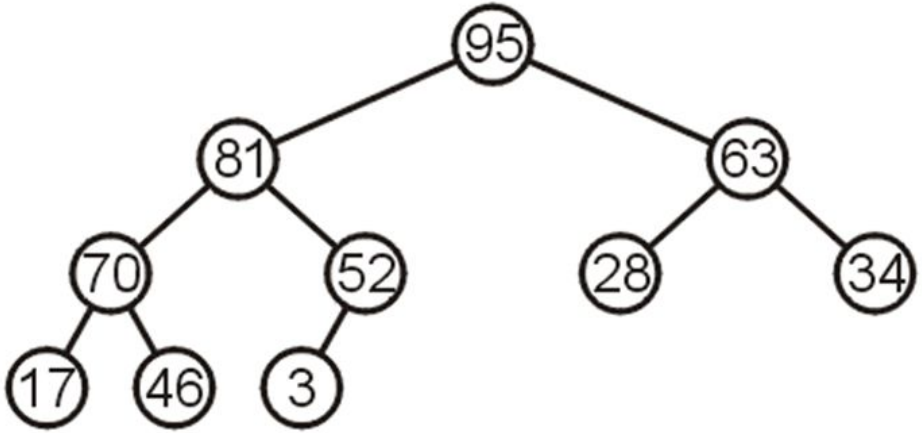
Heap Sort Example

We have now converted the unsorted array



into a max-heap:

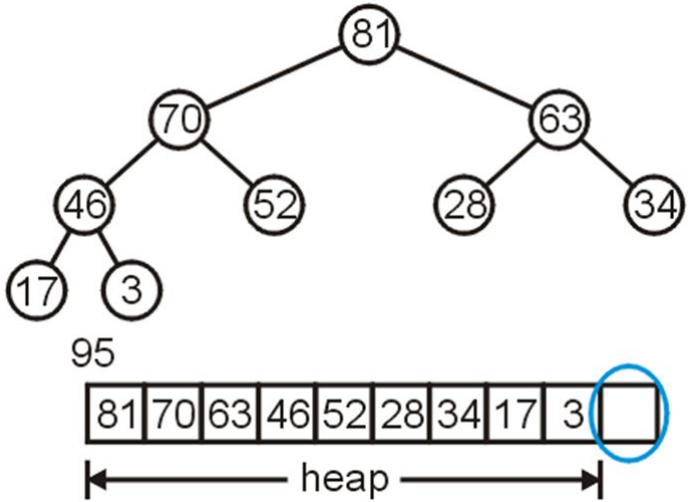




Heap Sort Example

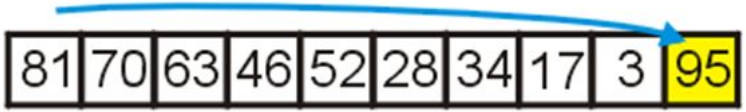
Suppose we pop the maximum element of this heap

This leaves a gap at the back of the array:

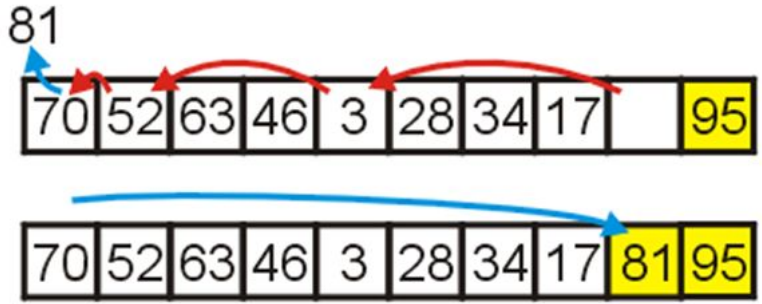


Heap Sort Example

This is the last entry in the array, so why not fill it with the largest element?



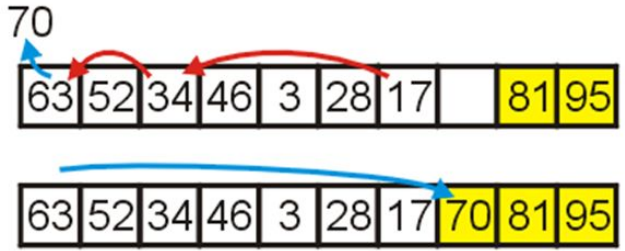
Repeat this process: pop the maximum element, and then insert it at the end of the array:



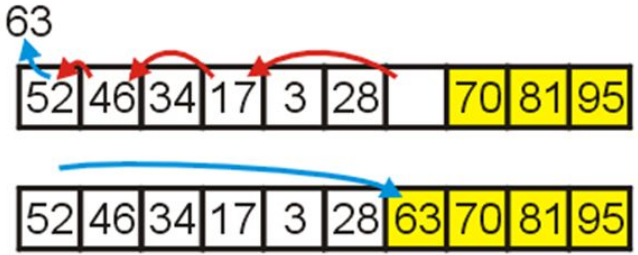
Heap Sort Example

Repeat this process

Pop and append 70



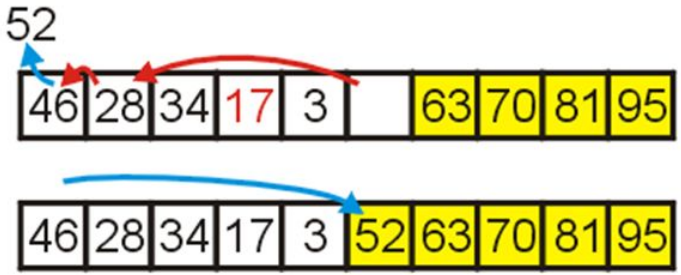
Pop and append 63



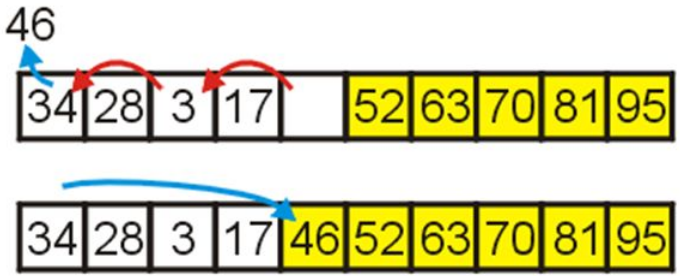
Heap Sort Example

We have the 4 largest elements in order

Pop and append 52



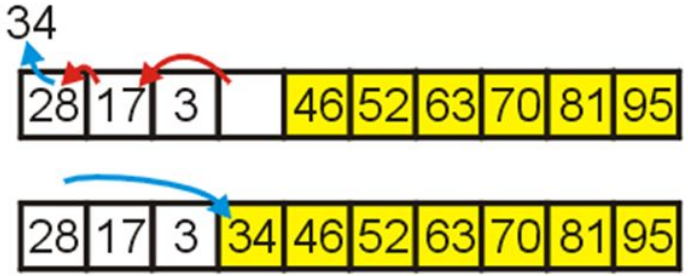
Pop and append 46



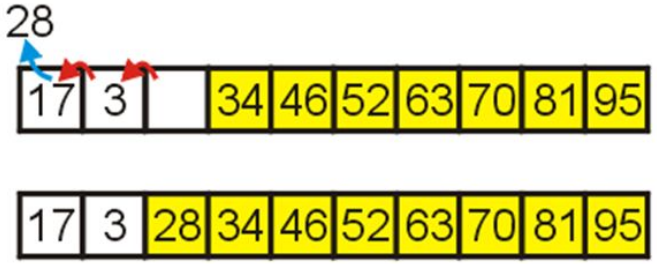
Heap Sort Example

Continuing...

Pop and append 34

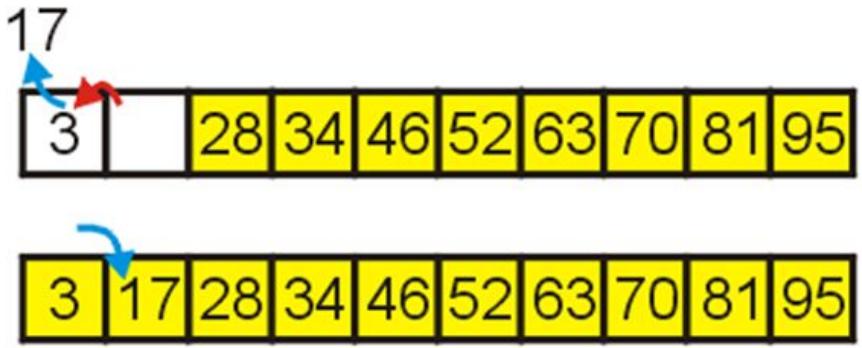


Pop and append 28



Heap Sort Example

Finally, we can pop 17, insert it into the 2nd location, and the resulting array is sorted



Runtime Summary

Heapification runs in $\Theta(n)$

Popping n items from a heap of size n, as we saw, runs in $\Theta(n \ln(n))$ time

We are only making one additional copy into the blank left at the end of the array

Therefore, the total algorithm will run in $\Theta(n \ln(n))$ time

End of Lecture